

Distributed Microservices Based E-Commerce Platform

ABSTRACT

Objective:

The main objective of this project is to develop a **secure, scalable, and efficient online shopping platform using Microservices Architecture**. The system is designed to overcome the limitations of traditional monolithic e-commerce applications by dividing the application into independent services. Each service handles a specific business functionality such as user management, product management, shopping cart, orders, discounts, payments, shipping, and returns. The platform allows customers to browse products, manage their shopping cart, place orders, make payments, track orders, and request returns. Administrators/vendors can manage products, categories, inventory, users, orders, and promotional offers.

Existing System:

The existing e-commerce applications are mostly developed using a **Monolithic Architecture**, where all modules such as user management, product catalog, shopping cart, payment, and order processing are combined into a single application.

The major limitations of the existing system are:

- Difficult to maintain and update.
- Limited scalability during heavy user traffic.
- Failure in one module may affect the entire application.
- Difficult to deploy individual modules independently.
- High maintenance cost.
- Difficult for multiple developers to work on different modules simultaneously.
- Technology upgrades become complex.
- Testing and debugging a large application can be difficult.

Proposed System:

The proposed system introduces a **Distributed Microservices-Based E-Commerce Platform** in which the application is divided into independent services. Each service performs a specific business function and communicates with other services using **REST APIs**.

The major services include:

- **User Service** – Handles user registration, login, authentication, and profile management.
- **Catalog Service** – Manages products, categories, pricing, and inventory.

- **Shopping Cart Service** – Handles adding, updating, and removing products from the shopping cart.
- **Discount Service** – Manages coupons, discounts, and promotional offers.
- **Order Service** – Handles order creation, cancellation, order history, and order status.
- **Payment Service** – Handles payment processing and payment status.
- **Shipping Service** – Manages shipment and delivery status.
- **Return Service** – Handles customer return requests.

The backend is developed using **Python with Django and FastAPI**, and **MySQL** is used for database management. The frontend is developed using **HTML, CSS, and JavaScript**.

The proposed system provides better **scalability, maintainability, flexibility, fault isolation, and easier development** compared with a traditional monolithic e-commerce application.

Users & Responsibilities:

The system consists of two main types of users:

- **Customer**
- **Admin/Vendor**

Customer

The customer can:

- Register and log in securely.
- Manage their profile.
- Browse and search products.
- View product details.
- Add or remove products from the shopping cart.
- Update product quantities.
- Apply coupons and discounts.
- Place orders.
- Make payments.
- View order history.
- Track order status.

- Cancel eligible orders.
- Request product returns.
- Add product reviews.

Admin/Vendor

The admin/vendor can:

- Manage user accounts.
- Add, update, and delete products.
- Manage product categories.
- Manage inventory.
- Manage product prices.
- Create and manage discounts and promotional offers.
- View and manage customer orders.
- Update order and shipment status.
- Manage returns.
- Maintain the overall system.

Hardware Requirements

- Processor: Intel Core i3 / AMD equivalent or above
- RAM: 8 GB or above
- Storage: 256 GB SSD or above
- Internet Connection: Required

Software Requirements

- Operating System: Windows 10/11 or Linux
- Programming Language: Python
- Backend Framework: Django
- API Framework: FastAPI
- API: REST API
- Database: MySQL
- Frontend: HTML5, CSS3, JavaScript
- IDE: Visual Studio Code / PyCharm
- API Testing: Postman
- Version Control: Git and GitHub
- Web Browser: Google Chrome / Microsoft Edge